

Context That Pays: Ablation-Calibrated Marginal Context Value Profiles for Budgeted Context Assembly

VENKATA SANGARAJU¹, (Member, IEEE), AND SUDHIR VISSA²

¹Independent Researcher, Leander, TX 78641, USA

²SAGE7 AI, Georgetown, TX 78628, USA

Corresponding author: Venkata Sangaraju (sangaraju1988@gmail.com)

ABSTRACT LLM agents assemble prompts from heterogeneous context (tool schemas, documents, examples, notes, history) under a token budget. We separate three questions: which context types matter for a model and task family, which blocks matter for an instance, and what to send under a budget. Marginal context value (MCV) profiles answer the first: the measured drop in success when a type is removed or shortened, plus pairwise type interactions, estimated offline by ablation with bootstrap intervals. A CP-SAT compiler maximizes predicted value under budget, dependency and variant constraints. In a pre-registered evaluation on Qwen3-4B-Instruct (HotpotQA, BFCL tool calling, synthetic history) the primary hypothesis failed: MCV-compiled context never significantly beat embedding relevance and trailed it by 0.053 AUBC on history. Two causes combined. Single-block deletions were too sparse to train a block ranker, which fell back on recency and length; and a note valued at zero because history duplicates it in the full context was dropped at small budgets, where it could still supply the answer. At a non-binding 8k budget MCV pruned 34.9% of tool-calling tokens with non-inferior success, but a post-hoc relevance ranker with a calibrated stopping rule pruned 83.4%, so the savings come from stopping, not from MCV. Neither a hybrid (profile admits types, relevance ranks blocks) nor interaction terms improved selection. As diagnostics the profiles were informative: value concentrated in tool schemas, documents and history, worked examples were worth little, and shortened tool schemas lost most of their value. Code, pre-registration and all logs are released.

INDEX TERMS Context engineering, LLM agents, prompt assembly, token budgets, counterfactual ablation.

I. INTRODUCTION

An LLM agent’s prompt is assembled, not written. Before each decision a harness collects tool schemas, retrieved documents, worked examples, notes and conversation history, and must fit them into a token budget set by latency, cost or the model’s window [1]. What to keep is usually decided by heuristics: a relevance score against the goal, a coverage objective over candidate passages [2], a compressor that deletes tokens [3], or fixed priorities written by an engineer. None of these asks the question a budgeted system actually faces: *how much does this kind of context change task success for this model on this task family?* The answer is not obvious. Models use long contexts unevenly [4], and a block can be relevant to the goal yet add nothing the model needs, or be redundant with another block that is already included.

This paper asks: when context consists of heterogeneous types, can we measure how much each type contributes to task success, and can those measurements be reused for budgeted assembly? We separate three levels of the problem (Section III): *type value* (which kinds of context matter for a model and task family), *block relevance* (which particular blocks matter for an instance), and *budget allocation* (what to send under a token limit, and when to stop). A *marginal context value (MCV) profile* answers the first level. For a (model, task family) pair it records the

measured drop in success when every block of a type is removed, the value of the type’s short variant, and pairwise interactions between types, each with a bootstrap interval. Profiles are estimated once, offline, by counterfactual ablation on development instances, with a length-neutral control. For the second level we train a block-value model on leave-one-block-out deltas from the same runs. For the third, a CP-SAT compiler maximizes predicted value under the budget, dependencies and variants without calling the language model, and reports a certificate of optimality for that predicted objective.

Prior work values context in related ways but for other decisions: attribution methods explain one generation by ablating its sources [5], [6], Shapley-style methods value demonstrations, prompts, data points or skills [7]–[10], and recent context-allocation studies measure the causal effect of context in generative search [11]. The distinguishing combination is a reusable, type-level, interval-reported value table measured per model and task family and compiled under a hard token budget.

We evaluate the approach under a pre-registered protocol (hypotheses, margins, sample sizes and the method version fixed before any test run, recorded as a commit in the released repository, whose run manifests name the commit each run descends from; it was not deposited with an external registry) on three task families: multi-hop question answering over HotpotQA documents with

distractors, tool selection and argument filling over a 30-tool catalog built from BFCL, and recall of the latest value of a user attribute from 24 synthetic sessions. All models run locally (Qwen3-4B-Instruct for the main study, Qwen3-8B for transfer). We compare against six baselines, including LLMLingua-2 and a PACMS-style coverage selector.

The primary result is negative, and we lead with it. **MCV did not beat relevance at block selection:** MCV-compiled context did not significantly exceed embedding relevance (the best baseline) in area under the success-budget curve in any family, and was 0.053 lower on history. Two causes combine there. Single deletions rarely change the outcome, so the block-value model learned proxies such as recency and length; at the smallest history budget it kept the session holding the answer in 64% of plans, against 94% for relevance. And the profile, measured by removal from the full context, valued a note that duplicates history at zero, so the compiler dropped the fallback that would have answered when that session was missing. Type values are local to the context in which they are measured. **Efficiency held, but is not specific to MCV.** The efficiency hypothesis, added after development pilots but before any test run, held in 2 of three families: at an 8k budget, which full contexts never reach, MCV pruned 34.9% of prompt tokens on tool use and 6.9% on history with non-inferior success. A pre-specified post-hoc follow-up showed that relevance with a calibrated stopping rule pruned more (83.4% on tool use, also non-inferior), and that a hybrid in which the profile admits types and relevance ranks blocks did not beat relevance. Interaction terms had no detectable effect. **As a diagnostic, the profiles were informative.** Tool schemas, documents and history carried most of the value (0.85, 0.35, 0.40), worked examples little on their own, and shortened tool schemas lost most of theirs. History and the profile note substituted for each other (interaction -0.60), and the length-neutral control moved no value by more than 0.05. Qwen3-4B and Qwen3-8B type values were rank-correlated ($\rho = 0.88$), but outcome agreement under transferred and native profiles may reflect compiler insensitivity as much as transfer.

Contributions.

- *A three-level view of context assembly* (type value, block relevance, budget allocation), with evidence that the levels should not be served by one signal: measured type value was informative, while block selection was best served by embedding relevance (Sections III and VI).
- *MCV profiles as a reusable type-level diagnostic:* an estimator of per-(model, family) type values, short-variant values and interactions with shrinkage, bootstrap intervals and a length-neutral control, with measured stability (ten to twenty development instances in subsampling), cost and Qwen3-4B-to-8B agreement.

- *A controlled, pre-registered evaluation* of ten context policies on three task families, including a failed primary hypothesis, a pre-specified follow-up, evidence that leave-one-block-out deltas were too sparse to train a block ranker that improves on embedding similarity, and evidence that full-context type values mislead allocation at small budgets for types that are redundant only in the presence of others.
- *A dependency- and variant-aware exact budget compiler* whose certificate is bounded to the predicted objective, not task success (Section IV).
- *An open artifact:* code, data adapters, logs of every model call, and scripts that regenerate every number, table and figure.

II. BACKGROUND AND RELATED WORK

A. CONTEXT ENGINEERING AND ASSEMBLY

A survey formalizes context engineering as the discipline of retrieving, processing and managing the information supplied to a language model at inference time [1]. Agentic Context Engineering treats contexts as evolving playbooks that accumulate and refine strategies [12], and DSPy compiles declarative pipelines of language-model calls into optimized prompts [13]; both optimize the *content* of context rather than selecting among existing blocks under a budget. Several 2026 systems assemble context for long-horizon agents. ContextPipe organizes assembly as a five-phase, database-style pipeline with a data-source catalog and a cache-aware optimizer, and notes that maximizing utility under a budget with precedence is a precedence-constrained knapsack that it deliberately does not search [14]. ContextRender selects earlier tool results using observed reuse in an execution-dependency graph together with recency and relevance [15]. Context Assembly as the Controlled Variable frames the choice of template, demonstrations and retrieval as the variable a harness controls [16]; ContextEvo learns a context-management policy from long-horizon trajectories [17]; programmatic context management treats context as an environment the agent manipulates [18]; and Imprompt treats prompts as a programmable control surface [19]. These systems decide what enters context through rules, learned policies or relevance signals. None measures, ahead of time, what each kind of block is worth to task success.

B. SELECTION UNDER A TOKEN BUDGET

PACMS casts assembly-time selection as submodular maximum coverage under a knapsack constraint and solves it with a cost-aware greedy over pluggable selectors [2]. Budgeted skill selection has been formulated as regularized submodular maximization with provable bicriteria guarantees, modelling complementarity through coverage and diminishing returns; its authors name interaction terms learned from execution feedback as future work [20]. In retrieval-augmented generation, budget allocation has been

approached by calibrated decisions about how much to retrieve [21], by a reader-context diagnostic for budget-constrained selection [22], by dependency-structured memory selection [23], and by constructing knowledge-base context from historical usage for text-to-SQL [24]. Tool schemas compete with retrieved evidence for the same window, which motivates schema compression [25], and budget-tiered protocols evaluate memory strategies for local agents [26]. Long contexts are also used unevenly by models, which perform worse when relevant information sits in the middle [4].

C. COMPRESSION

LLMLingua-2 distills token-level compression into a small classifier that keeps the tokens a teacher marks as essential [3], and COMI compresses context by a marginal-information-gain score that rewards relevance and penalizes redundancy [27]. A pre-registered randomized trial of compression in production task orchestration shows that savings depend on how compression changes output length [28], and compressing the persistent instructions of tool-using agents traces reliability frontiers [29]. Compression keeps every block but shortens it; our compiler instead chooses which blocks, and which variant of each, to include.

D. VALUING CONTEXT BY COUNTERFACTUALS

Valuing a unit by what its removal costs has a long history in data valuation, e.g. Data Shapley [9], and has been applied to demonstrations [7] and prompts [8]. For context, ContextCite attributes a generation to context sources with a surrogate model [5] and AttriBoT approximates leave-one-out attribution efficiently [6]; both are post-hoc explanations of one output. Closer to assembly, influence-guided context selection uses leave-one-out influence of retrieved passages in RAG [30]; the Laws of Context Allocation replace relevance proxies with a causal leave-one-out probe of generative reliance [11]; Decision-Aware Memory Cards rank candidate units by their estimated effect on the agent’s next action using judges or surrogates [31]; DRSR learns set-level deletion risk for agent history [32]; McClear clears memories with negative utility via cooperative attribution [33]; and SkillsInjector constructs skill context dynamically [34]. The closest work, SkillSV, values the internal units of a single skill document with structure-aware Shapley values computed from agent rollouts, accounting for dependencies between units [10]. Kernel surrogate models capture second-order interactions in task attribution [35]. An ablation of persistent context files for coding agents found no measurable effect on correctness [36], a reminder that block value must be measured rather than assumed.

E. POSITIONING

All counterfactual methods above value *one kind* of unit (passages, demonstrations, memories, history, or the parts

of one skill), *per instance*, for one model. We do not claim to be first to value context by its removal; influence-guided selection [30] and causal context allocation [11] already select context by leave-one-out effects at assembly time, per instance. Our contribution is a reusable, type-level value model across heterogeneous blocks (goal, instructions, tool schemas, documents, examples, history, notes) with cross-type interaction terms, a test of whether it transfers from a small profiling model to larger ones, and an exact dependency-aware compiler that uses it under a hard budget. The distinguishing element is reusable type-level value estimation across heterogeneous context classes, conditioned on model and task family, with uncertainty estimates, and compiled under explicit budget, dependency and variant constraints. We make no claim beyond that combination, and in our evaluation the type-level component did not by itself improve block selection over relevance ranking.

III. PROBLEM FORMULATION AND THREAT/FAILURE MODEL

A. BLOCKS, BUDGETS AND VALUE

Definition 1 (Typed context block). A block $b = (\tau_b, \{x_b^v\}_v, D_b)$ has a type τ_b from a fixed set $\mathcal{T}$ (goal, instructions, tool schema, skill document, fact, example, history turn, note), one or more textual variants x_b^v (full and optionally short), and a set D_b of blocks it depends on. Its cost $c_m(b, v)$ is the number of tokens of its rendered variant under model m ’s tokenizer.

An instance q of a task family f is a set of blocks $\mathcal{B}_q$ together with a deterministic success check. A selection S assigns to each block either a variant or nothing; it is *admissible* under budget B if every mandatory block (goal and instructions) is included, every included block’s dependencies are included, at most one variant per block is chosen, and $\sum_{b \in S} c_m(b, S(b)) \leq B$. A fixed, documented renderer turns S into one prompt, so success $Y_m(q, S) \in \{0, 1\}$ is a function of the selection and the model’s decoding.

Definition 2 (Marginal context value profile). For model m and family f , the MCV of type τ is $\text{MCV}(\tau) = \mathbb{E}_q[Y_m(q, \mathcal{B}_q) - Y_m(q, \mathcal{B}_q \setminus \tau)]$, the expected loss in success when every block of type τ is removed from the full context. The pairwise interaction of types τ_1, τ_2 is $I(\tau_1, \tau_2) = \mathbb{E}_q[Y(\mathcal{B}) - Y(\mathcal{B} \setminus \tau_1) - Y(\mathcal{B} \setminus \tau_2) + Y(\mathcal{B} \setminus \{\tau_1, \tau_2\})]$, positive when the two types are complements and negative when they are redundant or in conflict. The short-variant value $\text{MCV}_{\text{short}}(\tau) = \mathbb{E}_q[Y(\mathcal{B}^{\tau \rightarrow \text{short}}) - Y(\mathcal{B} \setminus \tau)]$ is the gain in success from rendering type τ short rather than removing it, so $\text{MCV}_{\text{short}}(\tau)/\text{MCV}(\tau)$ is the share of the type’s value that the short variant retains. A profile is the table of these quantities with confidence intervals. All three are local quantities: they are measured by removing types from the full context, and a type that is redundant there

can still matter when the context that substitutes for it is not sent (Section VI).

B. THREE LEVELS OF THE ASSEMBLY PROBLEM

Budgeted context assembly combines three questions that need not share one signal. *Level 1, type value*: which kinds of context matter for this model and task family? This is $MCV(\tau)$, a *measured* quantity: an average over development instances of an observed change in success. *Level 2, block relevance*: which particular documents, tools or sessions matter for this instance? Instances contain many blocks of one type (dozens of tool schemas or documents), so a type value cannot answer this; it needs a *predicted* per-block value $\hat{v}_{b,v}$, here either a learned block-value model or embedding similarity to the goal. *Level 3, budget allocation*: given these estimates, what should be sent under budget B , and when is more context not worth sending? The compiler answers Level 3 by maximizing $\sum \hat{v}_{b,v}$ (plus measured type interactions) subject to admissibility. This is a knapsack with dependency constraints, variant choice and pairwise type-presence terms; it is NP-hard in general, and with negative interactions the objective is not submodular, so greedy guarantees for coverage objectives [2], [20] do not apply. A recurring finding of Section VI is that a signal that is informative at Level 1 need not be informative at Level 2.

C. WHAT IS AND IS NOT GUARANTEED

Given its inputs, the compiler returns an admissible selection that is optimal for the predicted objective whenever the solver reports optimality, and it fails loudly if the mandatory blocks alone exceed the budget; it never silently truncates. Admissibility is defined on the sum of block costs; because tokenization is not additive across block boundaries, the rendered prompt can differ from that sum by a few tokens (Section VI reports the overshoot). Nothing guarantees that predicted value equals realized success: profiles are estimated from finite ablations on a development split, block values are predicted from features, and transfer to another model is an empirical question. The solver’s certificate is therefore a statement about the predicted objective under the constraints, never a statement about task success. These are exactly the claims Section VI tests.

IV. DESIGN

Figure 1 shows the pipeline: a typed block IR feeds an offline profiler that runs counterfactual ablations on a development split; the estimator turns outcomes into an MCV profile and an instance-level block-value model; at inference time the compiler solves a budgeted program over the blocks of each instance; the agent receives the rendered selection.

A. TYPED BLOCK IR AND RENDERING

Each instance is converted by a deterministic adapter into typed blocks with full and, where meaningful, short

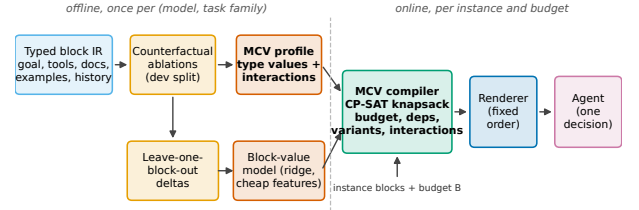


FIGURE 1. MCV profiles and the MCV compiler. Profiling happens once per (model, task family) on development data; compilation happens per instance and per budget without calling the language model (block embeddings are cached).

variants (a tool’s name and description without its parameter schema; the first sentence of a document; the first sentence of a history session). Rendering is a pure function of the selection: blocks appear in a fixed type order (instructions, tool schemas, skill documents, examples, documents, notes, history in chronological order, and the goal last), so identical selections yield byte-identical prompts and every policy in Section VI is evaluated through the same renderer. Token costs are measured with each model’s own tokenizer.

B. PROFILING BY COUNTERFACTUAL ABLATION

For each development instance the profiler evaluates the full context and a set of ablations: every optional type removed in turn; every pair of optional types removed; each type rendered with its short variant; and a sample of single blocks removed (leave-one-block-out), which trains the instance-level model below. A length-neutral control replaces a removed type with filler text of approximately equal token length, separating the effect of content from the effect of prompt length. All runs use greedy decoding with a fixed seed and are cached by prompt hash, so repeated conditions cost nothing.

C. ESTIMATION

Type values, short-variant values and interaction terms are paired means over development instances (Definition 2), each shrunk toward zero by $n/(n + n_0)$ for n instances, which damps cells measured on few instances, and reported with paired percentile bootstrap intervals. Instance-level block values come from a ridge regression trained on the leave-one-block-out deltas. Its features never use gold labels: block type, the type’s MCV, the fraction of the goal’s content words that occur in the block, the block’s BM25 rank within its type, log token length, for history its recency, and the cosine similarity between block and goal embeddings. A short variant’s value is the full value scaled by the type’s measured ratio $MCV_{\text{short}}(\tau)/MCV(\tau)$, clipped to $[0, 1]$.

TABLE 1. A compiled plan (tool family, Qwen3-4B-Instruct profile, 1k budget): the highest-valued optional blocks, any other chosen blocks, and the two lowest-valued blocks, with predicted value $\hat{v}$ and the chosen variant.

Block	Type	Tokens	$\hat{v}$	Chosen
calculate_future_value	tool schema	162	0.447	full
calculate_compound_interest	tool schema	169	0.339	full
calculate_stock_return	tool schema	174	0.312	full
kinematics.calculate_displacement	tool schema	177	0.106	short
biological.calc_biomass	tool schema	102	0.103	full
finance.calculate_quarterly_dividend_per_share	tool schema	112	0.099	full
humidity_temperature_forecast	tool schema	107	0.080	short
ex2	example	49	0.066	full
identify_bird	tool schema	126	-0.026	–
ex0	example	52	-0.042	–

33 optional blocks; 9 chosen; 996 of 1000 tokens; status OPTIMAL, 4.8 ms

D. THE MCV COMPILER

For instance q and budget B the compiler solves

$$\begin{aligned}
& \max \sum_{b,v} \hat{v}_{b,v} x_{b,v} + \sum_{\tau_1 < \tau_2} I(\tau_1, \tau_2) y_{\tau_1 \tau_2} \\
& \text{s.t. } \sum_v x_{b,v} \leq 1, \quad \sum_v x_{b,v} = 1 \text{ for mandatory } b, \\
& \quad \sum_v x_{b,v} \leq \sum_v x_{d,v} \quad \forall d \in D_b, \\
& \quad \sum_{b,v} c(b, v) x_{b,v} \leq B, \\
& \quad z_\tau = \max_{b: \tau_b = \tau} \sum_v x_{b,v}, \quad y_{\tau_1 \tau_2} = z_{\tau_1} z_{\tau_2},
\end{aligned}$$

with binary variables, using the CP-SAT solver of OR-Tools under a time limit. Values are scaled to integers. The solver returns a certificate (status, objective, bound, time); if no feasible solution is found in time, a value-density greedy fills the budget and the certificate records the fallback. The additive objective never gains from a block with non-positive predicted value, so the compiler may leave budget unused: more context is not assumed to be better. With interaction terms, a block of low or negative individual value can still be selected when its type completes a positive interaction. The additive variant (OURS-A) drops the interaction terms.

Table 1 shows one compiled plan from the test split: the predicted block values, the variant chosen for each block, and the certificate. The compiler takes the full schemas of the highest-value tools, falls back to the short variant (name and description only) for tools of moderate value where the full schema would not fit, and omits blocks whose predicted value is negative.

E. TRANSFER

A profile measured on a small model is applied unchanged to a larger one (OURS-T). Fidelity is reported as the rank correlation between the two models’ type values and as the success difference between transferred and natively measured profiles; the instance-level block-value model is shared, so the comparison isolates the type-level profile.

V. IMPLEMENTATION

The reference implementation is an open-source Python package (import name `mcv`, not published on PyPI, where that name belongs to an unrelated project; 1,267 lines; MIT license), with unit tests covering the edge cases the design must handle: budgets smaller than the mandatory

blocks (which raise an error rather than truncating), dependency cycles and unknown dependencies, zero-value blocks, deterministic rendering order, the solver’s timeout path, and estimator recovery of known effects (18 tests, 96% line coverage of the core modules). Models run locally through Ollama with greedy decoding, a fixed seed and a fixed context window, so that the model is never reloaded between conditions; models answer in a strict text protocol (a JSON call for tools, an ANSWER: line otherwise) parsed by the harness. Responses are cached by a hash of (model, seed, decoding options, prompt), so a prompt produced by several policies is answered once and the reuse is logged. Token costs use each model’s Hugging Face tokenizer, and the measured token count of every rendered prompt is logged so that budget compliance can be audited. Embedding-based baselines use the open nomic-embed-text model [37] through the same server. The compiler uses the CP-SAT solver of OR-Tools with a time limit per instance and a single worker for determinism. Every run writes JSON-lines logs of each call, selection and solver certificate, plus a manifest with the code commit, model digests and data hashes.¹

VI. EVALUATION

A. SETUP

Task families. Each family has 300 instances, split once by a seeded shuffle into 60 development and 240 test instances, with test hashes locked before any test run. *Facts:* HotpotQA distractor questions (validation split) [38]; each instance carries its native paragraphs plus extra distractors drawn from other questions (20 documents, 2,961 tokens on average over all instances), two worked examples, and asks for a short answer. *Tool:* a call from the Berkeley Function Calling Leaderboard (BFCL v3) of the Gorilla project [39] (simple and multiple categories); the correct function or functions are mixed into a 30-tool catalog (4,235 tokens), and success is an AST-level match of the emitted call. *History:* 24 synthetic chat sessions (2,718 tokens) in which one user attribute is set and later updated, a knowledge-update setting also probed by Long-MemEval [40]; the agent must report the latest value, and in half of the instances a note restates the *stale* value. Success checks are deterministic.

Models. Qwen3-4B-Instruct-2507 (Ollama tag `qwen3:4b-instruct`, a non-thinking release) is the main model and the profiling source; Qwen3-8B (tag `qwen3:8b`, the earlier hybrid-thinking release, run with thinking disabled) is the transfer target, so transfer spans both model size and post-training release. Both are the default 4-bit Ollama builds (model digests in the run manifests) and run locally with greedy decoding, a fixed

¹Software and data: Ollama 0.35.0; OR-Tools, scikit-learn, LLMingua and PyTorch versions pinned in `code/uv.lock`; HotpotQA (CC BY-SA 4.0) and BFCL v3 (Apache-2.0) as recorded in `data/LICENSES.md`.

seed, a 16,384-token window and at most 96 output tokens.

Policies. B0 full context (no budget); B1 truncate oldest; B2 embedding relevance (nomic-embed-text cosine to the goal, fill in descending order); B3 LLMingua-2 [3] compressing all optional context to the remaining budget; B4 a PACMS-style facility-location coverage selector [2] (our re-implementation); B5 fixed type tiers; OURS-A, the MCV compiler without interaction terms; OURS, with them. All policies keep the mandatory blocks and share the renderer.

Protocol. Profiles use the first 40 development instances (4B, with 6 leave-one-block-out samples each) and 20 instances (8B, type level). Budgets are 1k, 2k, 4k and 8k tokens. The primary metric is the area under the success–budget curve (AUBC), normalized over log-budget, per instance. The main grid evaluates B0, B2, B4, OURS-A and OURS on the first 100 test instances per family; the weaker baselines B1, B3, B5 (well below B2 in dev pilots) use the first 30. Following the pre-registration, H1 compares OURS with the best of B1–B5 per family; H1b tests, at 8k, that OURS uses fewer tokens than B0 and the best baseline with success non-inferior to B0 (margin 0.05); H2 compares OURS with OURS-A pooled; H3 tests transfer. Intervals are paired percentile bootstraps (10,000 resamples); p-values are Wilcoxon signed-rank for H1 and H2 and a one-sided paired bootstrap for H1b (the plan fixed the CI rule but named no test; recorded as a deviation); H1, H1b and H2 form one Holm family. All 6,180 main episodes completed. Block costs are summed in the model’s tokenizer; rendered prompts exceeded the budget in 34 of 5,880 budgeted main-study episodes, by at most 2 tokens, because tokens merge at block boundaries.

B. MAIN RESULT: H1 IS NOT SUPPORTED

Table 3 summarizes all pre-registered tests. Table 2 and Fig. 2 give the main comparison. Embedding relevance (B2) was the best baseline in every family. OURS did not exceed it anywhere: the paired AUBC difference was -0.025 [-0.070, 0.018] on facts and 0.007 [-0.012, 0.027] on tool use, both indistinguishable from zero, and -0.053 [-0.072, -0.035] on history, a significant *loss* (Holm $p < 0.001$). H1 required a significant gain in two families; it fails.

On tool use the MCV compiler beat the non-relevance baselines by wide margins at small budgets: at 1k it reached 92% success against 0% for LLMingua-2, 7% for truncation and 62% for coverage, because it spends the budget on the tool schemas ranked most valuable and often drops examples. This advantage did not hold elsewhere: on history at 1k OURS (64%) trailed truncation (70%) and tiers (70%). Relevance ranking also concentrates the budget on goal-related blocks. Once a block’s value is dominated by whether it is about the goal, an embedding score already captures most of what the profile adds.

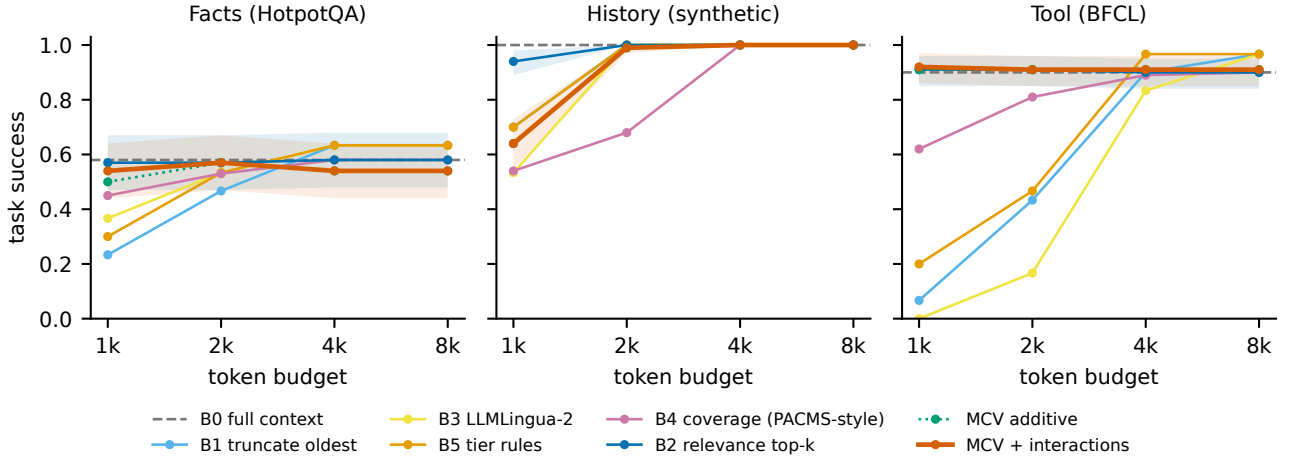
Diagnosis of the history loss. Almost all of the history gap arises at the 1k budget (at 2k and above both policies are at or near ceiling). There, success depends almost only on whether the session containing the latest update survives selection. OURS kept it in 64% of plans and B2 in 94%; without that session OURS never answered correctly (0%). Two causes combine. The first is the block-value model. Trained on leave-one-block-out deltas, its history coefficients (unstandardized; recency spans the unit interval, while goal similarity varies over a narrow range) weight recency (0.10) and token length (0.13) alongside embedding similarity (0.34), so it fills a small budget with the most recent sessions and misses the update whenever it lies further back. The second is the profile itself, applied far from where it was measured. The note’s value (0.00) is a leave-one-type-out effect from the full context, where history already supplies the answer, and the interaction term penalizes sending both. At 1k, OURS sent the note in 0% of plans, against 100% for relevance, so when the update session was dropped no fallback remained. In an exploratory check of the logs (not pre-registered), 18 of the 36 OURS failures at 1k were instances whose note held the *current* value, and policies that kept a current note but missed the update session answered correctly in 85% of 59 such episodes. A type value measured at full context is therefore not a valid value at a small budget for types that are redundant only in the presence of others. The leave-one-block-out signal is sparse: removing one of 24 sessions rarely changes the outcome, so the regression learns weak correlates of value instead of value. An exploratory check on the development logs confirms this (not pre-registered; five-fold cross-validation grouped by instance). Only 5% of history-session deletions changed the outcome (5% for documents, 4% for tool schemas). The cross-validated rank correlation between predicted and observed deltas was 0.15 for the ridge model against 0.27 for embedding similarity alone on history, 0.07 against 0.17 on facts, and 0.27 for both on tool use. The learned ranker was never better than its own similarity feature.

C. PRUNING: H1b IS SUPPORTED IN TWO OF THREE FAMILIES

Because the compiler adds blocks only while they raise the predicted objective, it leaves budget unused at 8k (Fig. 3), whereas B2 fills the budget and so matches full context. On tool use OURS sent 2,759 tokens instead of 4,240 (34.9% fewer) with success 91% against 90% for full context (difference 0.010, CI [0.000, 0.030], Holm $p < 0.001$). On history it sent 6.9% fewer tokens with identical success (100%), the saving coming from the examples, the note and a few low-value sessions. On facts it sent 30.9% fewer tokens, but success fell from 58% to 54% and the CI [-0.100, 0.010] crosses the margin, so non-inferiority is not shown there. Two families pass, as H1b required. Two caveats bound this result. Full contexts average 2,961, 2,718 and 4,235 tokens, so the 8k budget

TABLE 2. Test results on Qwen3-4B-Instruct: AUBC (mean, 95% paired-bootstrap CI) and success at the 1k and 8k budgets. B1, B3 and B5 use the first 30 instances per family, the others 100. B0 ignores the budget, so its 1k and 8k entries are the same full-context run.

Policy	Facts (HotpotQA)			History (synthetic)			Tool (BFCL)		
	AUBC	@1k	@8k	AUBC	@1k	@8k	AUBC	@1k	@8k
B0 full	0.580[0.480, 0.680]	58%	58%	1.000[1.000, 1.000]	100%	100%	0.900[0.840, 0.950]	90%	90%
B1 truncate oldest	0.511[0.372, 0.650]	23%	63%	0.950[0.922, 0.978]	70%	100%	0.617[0.528, 0.700]	7%	97%
B2 relevance	0.575[0.480, 0.667]	57%	58%	0.990[0.982, 0.997]	94%	100%	0.905[0.848, 0.957]	91%	90%
B3 LLMlingua-2	0.556[0.406, 0.706]	37%	63%	0.922[0.894, 0.950]	53%	100%	0.494[0.422, 0.567]	0%	97%
B4 coverage (PACMS-style)	0.542[0.453, 0.628]	45%	58%	0.817[0.773, 0.857]	54%	100%	0.820[0.758, 0.878]	62%	90%
B5 type tiers	0.544[0.394, 0.694]	30%	63%	0.950[0.922, 0.978]	70%	100%	0.672[0.583, 0.756]	20%	97%
OURS-A (MCV additive)	0.543[0.448, 0.637]	50%	54%	0.937[0.918, 0.953]	64%	100%	0.910[0.852, 0.960]	91%	91%
OURS (MCV + interactions)	0.550[0.455, 0.645]	54%	54%	0.937[0.918, 0.953]	64%	100%	0.912[0.852, 0.962]	92%	91%

**FIGURE 2.** Success versus token budget on Qwen3-4B-Instruct (test). Bands: 95% bootstrap CIs for B2 and OURS. The dashed line is full context. MCV additive and MCV with interactions overlap almost everywhere.**TABLE 3.** Pre-registered hypotheses. Differences are paired (OURS minus comparator); H1 and H2 in AUBC, H1b and H3 in success. p : Holm-adjusted over H1, H1b and H2 (two-sided Wilcoxon for H1 and H2, one-sided paired bootstrap for H1b); H3 is decided by its CI against the margin 0.05. Its interval is degenerate because transferred and native profiles gave identical outcomes in every pair, so H3 passes without being informative (see the transfer results below).

Hyp.	Family	Difference [95% CI]	p	Outcome
H1	Facts	-0.025 [-0.070, 0.018]	1.000	no gain
H1	History	-0.053 [-0.072, -0.035]	<0.001	worse
H1	Tool	0.007 [-0.012, 0.027]	1.000	no gain
H1b	Facts	-0.040 [-0.100, 0.010]	1.000	not shown
H1b	History	0.000 [0.000, 0.000]	<0.001	non-inferior
H1b	Tool	0.010 [0.000, 0.030]	<0.001	non-inferior
H2	Pooled	0.003 [-0.001, 0.007]	1.000	no effect
H3	Pooled	0.000 [0.000, 0.000]	–	non-inferior*

never binds (nor does 4k except on tool use). H1b therefore tests pruning of full context, not selection under a binding budget. The savings come from the block-level values, which stop adding blocks once predicted value reaches zero; the follow-up below shows that relevance with a stopping rule achieves them without MCV. On facts, full

context was right and OURS wrong on 6 instances, and the reverse held on 2. Only 2 of those losses removed a supporting paragraph; in the others the model changed its answer when distractors were removed, an effect a type-level profile cannot see.

D. INTERACTIONS: H2 HAS NO DETECTABLE EFFECT

Adding interaction terms changed pooled AUBC by 0.003 [-0.001, 0.007] (Holm $p=1.000$). History is at ceiling from 2k upward, which leaves little room for any difference, so this null result is limited. The only large interaction measured is between history and the profile note (-0.60). Because the note is current in half of the instances and stale in the other half, the negative sign means the two substitute for each other: either can supply the answer when it is current, and the note adds nothing once history is present (its own value is 0.00). The interaction terms did change the plans: OURS and OURS-A selected different blocks in 30% of facts, 50% of history and 22% of tool plans. At 8k, OURS never included the note while OURS-A included it in 49% of history plans, and the positive example–document interaction (0.10) made OURS keep an

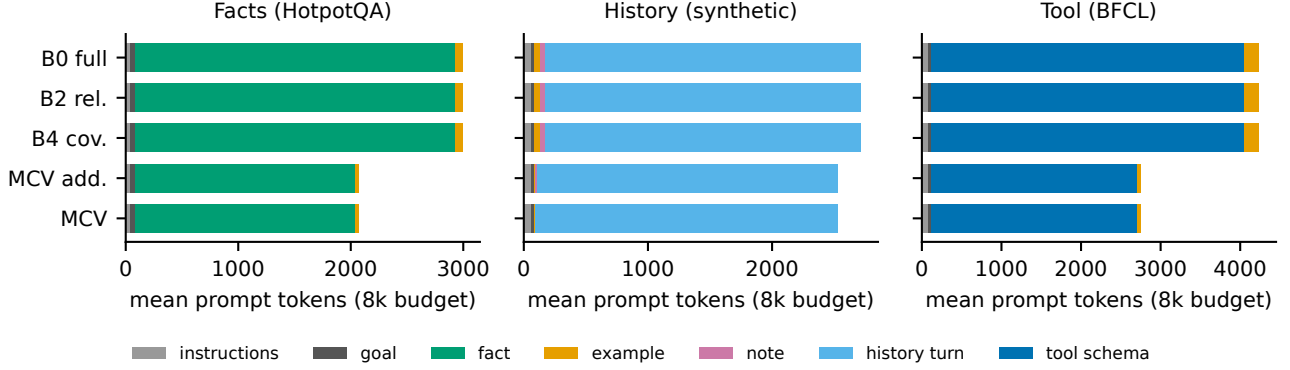


FIGURE 3. Mean prompt composition at the 8k budget (test, Qwen3-4B-Instruct). MCV leaves budget unused: it drops the profile note, most low-value documents and tool schemas, and some worked examples.

example in 100% of facts plans against 86% for OURS-A. These changes concern low-value blocks and rarely changed success (2.0%, 0.5% and 0.2% of episodes). On these families, type-pair interactions change what is sent but not whether the task succeeds.

E. POST-HOC FOLLOW-UP: STOPPING RULES AND A HYBRID

The results above leave two questions open: are the token savings specific to MCV, and does the three-level split of Section III help if each level uses the signal that worked? After the main results were known we pre-specified a follow-up (an addendum to the pre-registration, committed before any of its runs) with two policies. *B2S* is relevance ranking with a stopping rule: blocks whose goal cosine falls below a threshold τ are never added. *HYB* is a hybrid: the 4B profile admits only types whose MCV interval excludes zero (documents, history sessions, tool schemas), relevance ranks blocks within admitted types with value $\cos - \tau$, and the compiler applies the budget, dependencies and short variants. Each family’s τ was chosen from a five-point grid on 20 development instances not used for profiling, as the most token-saving value whose success at 8k stayed within 0.05 of full context. The same rule picked the same τ for both policies in every family. Both were run on the main test instances and budgets; H6 tests B2S and H8 tests HYB for non-inferiority to full context at 8k, H7 tests HYB against B2 in AUBC (one Holm family of nine tests).

Table 4 gives the results. **The hybrid did not improve selection.** HYB exceeded B2 in none of the three families (H7; Holm-adjusted p 1.000, 1.000 and 1.000), and its AUBC was close to that of OURS (HYB–OURS: -0.020, -0.002, -0.002). On history its bootstrap interval against B2 lies entirely below zero (-0.055 [-0.100, -0.018]), although the Wilcoxon test is not significant: its calibrated threshold sometimes excludes the update session itself, at every budget, the same loss B2S shows on history. **The type filter added nothing measurable here:** HYB and B2S

TABLE 4. Post-hoc follow-up (test, Qwen3-4B-Instruct, 100 instances per family). AUBC; success and prompt-token reduction relative to full context at 8k. Non-inferiority margin 0.05.

	Facts	History	Tool
AUBC: B2 / OURS	0.575 / 0.550	0.990 / 0.937	0.905 / 0.912
AUBC: B2S / HYB	0.522 / 0.530	0.937 / 0.935	0.920 / 0.910
H7: HYB–B2 [CI]	-0.045 [-0.132, 0.040]	-0.055 [-0.100, -0.018]	0.005 [-0.030, 0.040]
@8k success: B0 / OURS	58% / 54%	100% / 100%	90% / 91%
@8k success: B2S / HYB	52% / 53%	94% / 94%	92% / 91%
@8k tokens cut: OURS	30.9%	6.9%	34.9%
@8k tokens cut: B2S / HYB	73.9% / 75.2%	52.2% / 55.4%	83.4% / 85.6%
H6: B2S–B0 [CI]	-0.060 [-0.150, 0.030]	-0.060 [-0.110, -0.020]	0.020 [0.000, 0.050]

produced the same outcome in 99% of episodes, although B2S kept an example or the note in most of its plans. This is consistent with the profiles: the excluded types carry little value, so removing them saves some tokens (85.6% against 83.4% on tool use) without changing outcomes. **The savings are not specific to MCV.** On tool use, relevance with a stopping rule was non-inferior to full context at 8k (H6, Holm $p < 0.001$) while removing 83.4% of prompt tokens, against 34.9% for OURS; HYB behaved alike (H8, $p = 0.005$). On facts and history the calibrated thresholds were too aggressive for the test set: they removed 73.9% and 52.2% of tokens, but success fell by -0.060 and -0.060, and non-inferiority was not shown. There, the more conservative OURS kept success at the cost of smaller savings. A threshold calibrated on 20 instances is a coarse instrument; the facts calibration curve was not even monotone. On tool use the rule chose the largest threshold in the grid, so the saving reported there is a lower bound on what a stopping rule could remove.

F. WHAT THE PROFILES SAY

Fig. 4 and Table 5 show the profiles of both models. Three regularities hold across families. First, the value of context is concentrated: tool schemas (0.85), documents (0.35) and history sessions (0.40) carry nearly all of it. Second, worked examples are worth close to nothing on their own (0.05, 0.00, -0.03 on the 4B model; the 8B model valued them at 0.10 on history), although every

TABLE 5. MCV profiles on the development split: unshrunk paired means with 95% bootstrap CIs. Rows are type values (success drop on removal), short-variant values (gain of the short variant over removal) and pairwise interactions. The 8B model was profiled at type level on fewer instances, so its intervals are wider.

Family	Term	Qwen3-4B	Qwen3-8B
Facts	base success	0.53	0.55
	example	+0.05 [0.00, +0.12]	0.00 [0.00, 0.00]
	fact	+0.35 [+0.17, +0.53]	+0.35 [+0.10, +0.60]
	short fact	+0.12 [-0.03, +0.28]	+0.20 [0.00, +0.45]
	<i>I</i> (example, fact)	+0.10 [+0.03, +0.20]	0.00 [0.00, 0.00]
History	base success	1.00	1.00
	example	0.00 [0.00, 0.00]	+0.10 [0.00, +0.25]
	history turn	+0.40 [+0.25, +0.55]	+0.45 [+0.25, +0.65]
	note	0.00 [0.00, 0.00]	+0.05 [0.00, +0.15]
	short history turn	-0.03 [-0.12, +0.05]	0.00 [0.00, 0.00]
	<i>I</i> (example, history turn)	0.00 [0.00, 0.00]	+0.10 [0.00, +0.25]
	<i>I</i> (example, note)	0.00 [0.00, 0.00]	+0.10 [-0.10, +0.30]
	<i>I</i> (history turn, note)	-0.60 [-0.75, -0.45]	-0.50 [-0.70, -0.30]
Tool	base success	0.85	0.85
	example	-0.03 [-0.07, 0.00]	-0.05 [-0.15, 0.00]
	tool schema	+0.85 [+0.72, +0.95]	+0.80 [+0.60, +0.95]
	short tool schema	+0.30 [+0.17, +0.45]	+0.25 [+0.05, +0.45]
	<i>I</i> (example, tool schema)	+0.03 [-0.05, +0.10]	0.00 [-0.15, +0.15]

family’s prompt includes them by default. Third, short variants are worth much less than full blocks for tool schemas (0.30 against 0.85), because the parameter schema matters for argument filling. The length-neutral control, which replaces a removed type with filler of approximately equal token length (20 instances per type), reproduced the removal effects: documents 0.50 by removal against 0.45 with padding, tool schemas 0.90 against 0.90, history 0.45 against 0.45. The measured values reflect content, not prompt length.

G. TRANSFER ACROSS MODEL SIZE AND RELEASE (H3)

The 4B and 8B profiles rank the 7 (family, type) values similarly (Spearman $\rho = 0.88$, pooled across families, so between-family differences contribute to the agreement), and their magnitudes agree closely (Fig. 4). On Qwen3-8B (first 40 test instances per family, budgets 2k and 8k), compiling with the transferred 4B profile and with the natively measured 8B profile gave a pooled success difference of 0.000 [0.000, 0.000] over 240 paired episodes, so H3 non-inferiority holds (Fig. 5). The outcome was the same in 100% of paired episodes, giving the degenerate interval above, although the two profiles produced identical selections in only 35%. The differences involved tool schemas, history sessions and examples alike, yet never changed the outcome. This suggests that both profiles keep the blocks that decide success and differ only in blocks that do not. It equally indicates that, with the block-value model shared, the compiler’s outcomes are insensitive to the type-level profile, so H3 does not show that the profile drives the decisions that matter. The efficiency pattern of the 4B study carried over. On tool use at 8k the transferred profile sent 2,821 prompt tokens against 4,232 for relevance ranking,

with success 92% against 90%. Relevance ranking was again more accurate on facts (68% against 62% at 2k). With 40 instances per family these 8B comparisons are descriptive.

H. FAILURE ANALYSIS

We read the transcripts of failed episodes in each family (representative cases per failure mode are extracted verbatim from the logs by a script in the released repository). *History at small budgets*: when the compiler drops the session holding the update, the model does not guess. It reports that no conversation mentions the attribute, so these failures are abstentions rather than hallucinations. When B2 keeps the same session, the answer is correct. *Compression of identifiers*: LLMingua-2 deletes the underscores and reorders words in function and parameter names. The model then emits calls whose names do not exist, which is why B3 scores far below the selection policies on tool use at small budgets even though the call’s intent is right. Selection policies never alter block text, so they cannot fail this way. *Facts with all evidence present*: in several facts losses at 8k the supporting paragraphs were kept and only distractors were removed, yet the model’s answer changed: it answered a comparison question with a year rather than a title, or added detail that the exact-match checker rejects. Removing context changes the model’s behavior in ways a value table indexed by type cannot predict, and in these cases the change was harmful.

I. HOW MUCH PROFILING IS NEEDED?

Profiling cost scales with the number of development instances, so we asked, again as an exploratory analysis of the existing logs, how quickly the type-level profile stabilizes. We re-estimated the 4B profiles from random subsets of the 40 profiled instances (300 draws per size) and compared them with the full-sample profile (Fig. 6). With five instances per family the mean absolute error of the type values was 0.09 and the most valuable type was identified in all three families in 74% of draws. With ten instances these were 0.06 and 92%, and with twenty 0.03 and 99%. Because subsets are drawn from the same 40 instances, agreement at larger sizes is partly mechanical. The practical reading is that the type-level ranking needs on the order of ten to twenty instances per family, while the block-level model would need far more informative supervision than single-block deletions provide.

J. COST

Compilation. Over 2,400 compilations CP-SAT proved optimality in 100.0% of cases with no fallback (0); median solve time was 2.3 ms (99th percentile 22.3 ms, maximum 357 ms), and end-to-end planning including feature computation (with cached block embeddings) took a median 6.9 ms. **Profiling.** The 4B profile cost 1,440 model calls (3.0 hours on a laptop). Measured in prompt tokens, profiling pays for itself after about 931 queries on

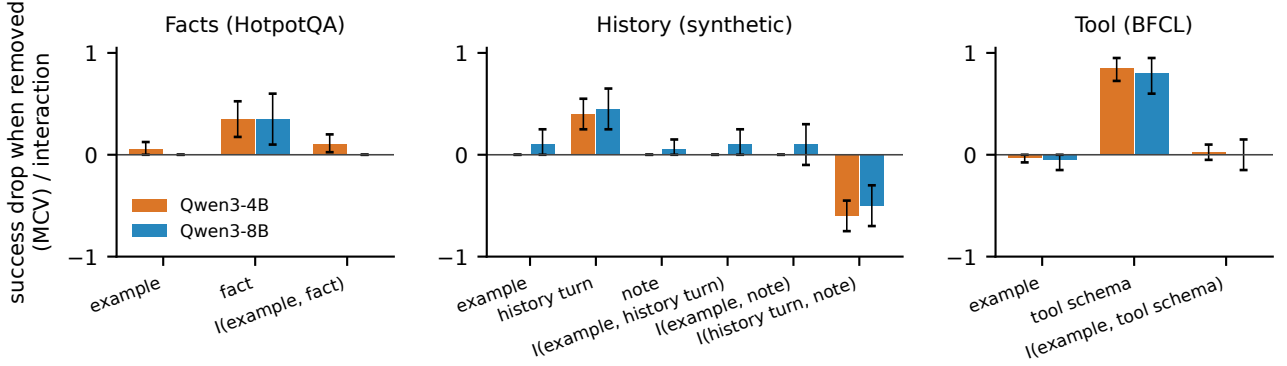


FIGURE 4. MCV profiles (development split): success drop when a type is removed, and pairwise interactions $I(\tau_1, \tau_2)$, for Qwen3-4B-Instruct (40 instances) and Qwen3-8B (20 instances), with 95% bootstrap CIs.

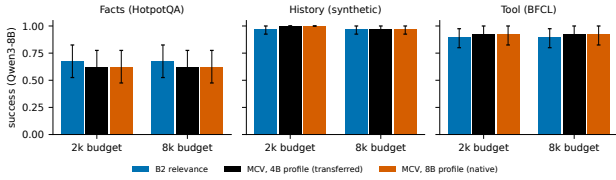


FIGURE 5. Transfer to Qwen3-8B (test, 40 instances per family): relevance ranking, MCV with the transferred 4B profile, and MCV with the native 8B profile, with 95% CIs.

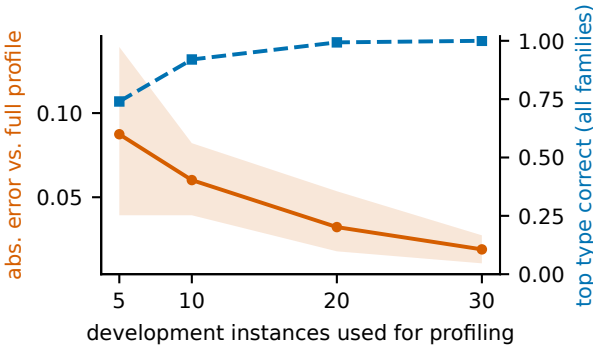


FIGURE 6. Profile stability versus development instances (exploratory; subsamples of the 4B profiling logs). Band: the range of the middle four fifths of draws.

tool use and 1,016 on facts, given the 8k-budget savings above. The facts figure is conditional on accepting that family’s non-significant success loss. History saves less per query and breaks even after about 5,901.

VII. DISCUSSION, LIMITATIONS AND THREATS TO VALIDITY

A. WHAT THE RESULTS MEAN

In these experiments measured context value was more useful for diagnosing which kinds of context matter than for choosing individual blocks. At the block level, embedding relevance was hard to beat: neither the MCV compiler

nor the hybrid improved on it, and the learned block ranker was worse where it substituted recency for relevance. At the type level, the profiles exposed structure that relevance scores do not report: value concentrated in one or two types per family, worked examples with little individual value, a substitution between history and the note, and a large loss from shortening tool schemas. The token savings came from not sending context, and a relevance ranker with a stopping rule achieved them as well, so they are a property of stopping rather than of MCV.

Type values have a limit of their own: they are local to the full context in which they are measured, so a type that is redundant there (the note) gets no value where it is the only carrier of the answer. Values estimated from random subsets of types and blocks, as sparse linear surrogates do for attribution [5], would address this and give denser block-level supervision than single deletions.

The block-level failure is informative about supervision. When many blocks are individually dispensable (one session of 24, one document of 20), single deletions rarely change the outcome, and a regression on those deltas learns proxies. Group or pairwise deletions, or relevance itself as the within-type ranker, are the natural fixes; the hybrid tested the latter and matched relevance with a stopping rule almost exactly.

B. DESIGN LESSON AND FUTURE WORK

The evidence suggests separating type-level valuation from instance-level relevance: a type profile decides which types deserve budget and flags low-value or redundant context; relevance ranks blocks within types; a stopping rule or constraint-aware compiler decides when more context is not worth sending. Our hybrid instantiated this pipeline and did not beat relevance on these families, where the excluded types already had little effect on outcomes. Whether the type level helps where low-value types are large, numerous or harmful remains untested. Three experiments would most strengthen these conclusions: (i) stopping thresholds calibrated on a larger development set,

since our calibration on 20 instances per family was too aggressive for facts and history; (ii) the same protocol on a model family other than Qwen, which would test whether profiles and the null selection result generalize; and (iii) tasks with binding budgets and long-horizon trajectories, where type-level allocation has more room to matter.

C. PRACTICAL GUIDANCE

Profile each (model, task family) on ten to twenty development instances (enough here to rank types) with type removals and the length-neutral control, and use the profile to audit the default prompt: drop types whose value interval sits at zero, after checking their interactions. Rank blocks by relevance, and calibrate a stopping threshold on held-out data rather than filling the budget.

D. LIMITATIONS AND THREATS TO VALIDITY

Construct validity. Each instance is a single agent decision, not a multi-step trajectory with a simulated user as in τ -bench [41]; context value in long-horizon agents may compound across steps. Success is a deterministic check (answer match, AST-level call match), which misses partially correct outputs. **Internal validity.** The block-value model went through three development versions (dev AUBC 0.633, 0.758, 0.736); the second was frozen by a rule fixed in advance before any test run, and all pilots are in the logs. B4 is our re-implementation of a PACMS-style objective, not the authors’ code, and B3 uses one LLMingua-2 checkpoint without protected tokens, so it can corrupt tool and argument names; both could be tuned further. Rendered prompts may exceed the budget by at most 2 tokens because tokenization is not additive across block boundaries. Reading transcripts after the test runs revealed a defect in the tool checker: it compared list-valued arguments as strings, so an integer list failed against the same values written as floats. Rescoring all logged answers with element-wise numeric comparison changed 22 episodes, spread across all policies, and left the H1, H1b and H2 mean paired differences unchanged at three decimals (for example H1 on tool use, 0.007). We keep the pre-registered checker as primary. Greedy decoding on Apple-silicon inference is not guaranteed to be bitwise deterministic; caching makes every reported response reproducible from the logs, but a fresh rerun may differ slightly. **Statistical conclusion validity.** The pre-registered power analysis put power for a 0.05 AUBC effect on facts at about 58% with 100 instances, so the null result on facts is weak evidence of no effect. The weaker baselines used 30 instances; each scored below B2 in every family (Table 2), so B2 is the best baseline throughout. The pre-registration is a commit in our own repository, not an externally time-stamped registry entry; before publication the history was rewritten to remove two private planning files, and a commit map in the repository links the original commit hashes recorded in every run manifest to the published ones. The follow-up (B2S, HYB)

was specified after the main results were known and reuses the same test instances; its thresholds were calibrated on 20 development instances per family, which proved noisy. **External validity.** Two models of one family (Qwen3, 4B and 8B) were tested; a third model from another family was planned and dropped for compute, so neither the profiles nor transfer are shown to generalize beyond Qwen3. Identical paired outcomes under transferred and native profiles, despite differing selections, cannot distinguish faithful transfer from compiler insensitivity. One task family (history) is synthetic; HotpotQA is known to admit reasoning shortcuts that MuSiQue was built to remove [42]; and HotpotQA and BFCL may overlap with pretraining data. Profiles are per (model, task family) and must be re-measured when either changes, at the profiling cost reported in Section VI. That break-even is computed against full context; relevance with a stopping rule reached larger savings without any profiling.

VIII. CONCLUSION

We asked whether measuring how much each type of context contributes to task success can be reused for budgeted context assembly. In these experiments, measured context value was more useful for diagnosing which kinds of context matter than for ranking individual blocks. Embedding relevance remained difficult to beat at the instance level: neither the MCV compiler nor a hybrid built on the profile improved on it, and supervision from single-block deletions was too sparse to learn block value. Type-level ablations nevertheless exposed concentrated, redundant and low-value context, and the largest token savings came from deciding when to stop sending context. These results motivate context-assembly designs that keep type-level valuation and block-level relevance separate, and we release every log so that the negative results can be checked and extended.

Use of AI assistance. An AI coding assistant was used to write and test code, run the experiments, and draft and edit text. All numbers in the paper are generated from the released logs by scripts, and every reference was verified against its DOI record. The authors reviewed the content and are responsible for it.

Artifact availability. The code, data adapters, pre-registration, logs of every model call, and scripts that regenerate all results with one command are available at <https://github.com/sage7AI-explore/context-value-profiles> (code under the MIT license; data under the licenses of their sources).

References

- [1] L. Mei, J. Yao, Y. Ge, Y. Wang, B. Bi, Y. Cai, J. Liu, M. Li, Z.-Z. Li, D. Zhang, C. Zhou, J. Mao, T. Xia, J. Guo, and S. Liu, “A Survey of Context Engineering for Large Language Models,” arXiv preprint arXiv:2507.13334, 2025.
- [2] M. Ghulyani, A. Singh, K. Bharadwaj, A. Nath, and S. Goswami, “PACMS: Submodular Context Selection as a Pluggable Engine for LLM Agents,” arXiv preprint arXiv:2606.20047, 2026.

- [3] Z. Pan, Q. Wu, H. Jiang, M. Xia, X. Luo, J. Zhang, Q. Lin, V. Rühle, Y. Yang, C.-Y. Lin, H. V. Zhao, L. Qiu, and D. Zhang, “LLMLingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression,” arXiv preprint arXiv:2403.12968, 2024.
- [4] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, “Lost in the Middle: How Language Models Use Long Contexts,” *Transactions of the Association for Computational Linguistics*, 2024.
- [5] B. Cohen-Wang, H. Shah, K. Georgiev, and A. Madry, “ContextCite: Attributing Model Generation to Context,” arXiv preprint arXiv:2409.00729, 2024.
- [6] F. Liu, N. Kandpal, and C. Raffel, “AttriBoT: A Bag of Tricks for Efficiently Approximating Leave-One-Out Context Attribution,” arXiv preprint arXiv:2411.15102, 2024.
- [7] S. Xie, M. Luo, C. D. Stern, M. Du, and L. Cheng, “DemoShapley: Valuation of Demonstrations for In-Context Learning,” arXiv preprint arXiv:2410.07523, 2024.
- [8] H. Liu, X. Mao, H. Xia, J. Lou, J. Liu, and K. Ren, “Prompt Valuation Based on Shapley Values,” arXiv preprint arXiv:2312.15395, 2023.
- [9] A. Ghorbani and J. Zou, “Data Shapley: Equitable Valuation of Data for Machine Learning,” arXiv preprint arXiv:1904.02868, 2019.
- [10] T. Li, J. Liu, Q. Zhao, Y. Li, L. Wang, B. Shao, X. Liu, and L. Shou, “What Is a Skill Worth? Structure-Aware Shapley Valuation of Agent Skills,” arXiv preprint arXiv:2608.04562, 2026.
- [11] P. Liu, X. Wang, D. Liang, and W. Ye, “The Laws of Context Allocation: Causal Measurement and Closed-Loop Orchestration in Generative Search,” arXiv preprint arXiv:2608.23252, 2026.
- [12] Q. Zhang, C. Hu, S. Upasani, B. Ma, F. Hong, V. Kamanuru, J. Rainton, C. Wu, M. Ji, H. Li, U. Thakker, J. Zou, and K. Olukotun, “Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models,” arXiv preprint arXiv:2510.04618, 2025.
- [13] O. Khattab, A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, S. Vardhamanan, S. Haq, A. Sharma, T. T. Joshi, H. Moazam, H. Miller, M. Zaharia, and C. Potts, “DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines,” arXiv preprint arXiv:2310.03714, 2023.
- [14] P. Xu, Z. Zhang, Y. Sun, F. Tian, L. Wang, and C. Zhang, “ContextPipe: Database-Inspired Context Assembly for Long-Horizon Agents,” arXiv preprint arXiv:2609.00749, 2026.
- [15] S. Kashmira, J. L. Dantanarayana, L. Tang, and J. Mars, “ContextRender: From Execution Dependencies to Agent Context,” arXiv preprint arXiv:2609.37743, 2026.
- [16] D. Paul, “Context Assembly as the Controlled Variable: A Control-Theoretic View of Harness Policies for Frozen LLM Agents,” arXiv preprint arXiv:2607.25408, 2026.
- [17] W. Li, J. Xu, A. Chen, X. Wang, S. Liang, J. Liang, and D. Yang, “Beyond Skill Evolution: Self-Evolving Context Management Policies for Long-Horizon Agent Harnesses,” arXiv preprint arXiv:2609.34649, 2026.
- [18] Y. Lin, E. Ang, E. Zhu, B. Ding, and J. Zhou, “Context as an Environment: Programmatic Context Management for Long-Horizon Agents,” arXiv preprint arXiv:2608.21690, 2026.
- [19] C. Wu, S. Yang, and A. Murali, “Imprompt: A Language Framework for Prompt Programming,” arXiv preprint arXiv:2607.22683, 2026.
- [20] Y. Chen, R. Chen, X. Wang, Z. Li, and L. Huang, “Optimal Skill Selection for LLM Agents with Provable Bicriteria Guarantees,” arXiv preprint arXiv:2608.19993, 2026.
- [21] Z. Dong, F. Qin, M. Shah, and Y. Wang, “Know Before You Fetch: Calibrated Retrieval-Budget Allocation for Retrieval-Augmented Generation,” arXiv preprint arXiv:2606.29959, 2026.
- [22] A. N. Bala, “Recall Is Not Enough: A Reader-Context Diagnostic for Budget-Constrained Retrieval-Augmented Generation,” arXiv preprint arXiv:2607.00725, 2026.
- [23] Y. Wu, Y. Zhang, S. Ghosh, S. Basu, A. Deoras, J. Huan, and G. Gupta, “ContextWeaver: Selective and Dependency-Structured Memory Construction for LLM Agents,” arXiv preprint arXiv:2604.23069, 2026.
- [24] K. Gwimm and C. Eisenach, “Beyond the Harness: End-to-End Optimization of Context Artifacts for Enterprise Text-to-SQL,” arXiv preprint arXiv:2608.22830, 2026.
- [25] F. Sakizli, “Tool-Schema Compression Enables Agentic RAG Under Constrained Context Budgets,” arXiv preprint arXiv:2605.26165, 2026.
- [26] A. K. G. Rao and A. Jaggi, “BudgetBench: A Budget-Tiered Protocol and Pilot Harness for Memory Strategy Evaluation in Local Large Language Model Agents,” arXiv preprint arXiv:2609.13149, 2026.
- [27] J. Tang, S. Liu, Z. Zhang, Y. Yuan, L. Zheng, W. Su, and B. Zheng, “COMI: Coarse-to-fine Context Compression via Marginal Information Gain,” arXiv preprint arXiv:2602.01719, 2026.
- [28] W. Johnson and C. Lee, “Prompt Compression in Production Task Orchestration: A Pre-Registered Randomized Trial,” arXiv preprint arXiv:2603.23525, 2026.
- [29] Y. Hou and Z. Yang, “Control Under Compression: Reliability Frontiers for Tool-Using Agents,” arXiv preprint arXiv:2608.01056, 2026.
- [30] J. Deng, Y. Shen, Z. Pei, Y. Chen, and L. Huang, “Influence Guided Context Selection for Effective Retrieval-Augmented Generation,” arXiv preprint arXiv:2509.21359, 2025.
- [31] X. Guan, Q. Zhao, and Y. Deng, “Decision-Aware Memory Cards: Counterfactual-Inspired Context Selection and Compression for Tool-Using LLM Agents,” arXiv preprint arXiv:2606.08151, 2026.
- [32] M. Wang, B. Wang, F. Luo, G. Yao, C. Ning, Y. Guo, H. Chen, Y. Ma, and J. Han, “DRSR: Learning Set-Level Deletion Risk for Efficient Long-Horizon Agents,” arXiv preprint arXiv:2609.27276, 2026.
- [33] B. Yang, J. Sun, Z. Lu, Z. Qiu, X. Peng, and J. Zheng, “McClear: Cooperative Game-Theoretic Attribution and Risk-Aware Memory Clearance for Long-Horizon LLM Agents,” arXiv preprint arXiv:2609.09115, 2026.
- [34] Y. Li, W. Liu, B. Gao, J. Xie, Z. Ai, N. Zou, Y. Li, and T. Fu, “SkillsInjector: Dynamic Skill Context Construction for LLM Agents,” arXiv preprint arXiv:2605.29794, 2026.
- [35] Z. Zhang, M. Duan, and H. R. Zhang, “Efficient Estimation of Kernel Surrogate Models for Task Attribution,” arXiv preprint arXiv:2602.03783, 2026.
- [36] P. Khatri, “Do Context Files Help Coding Agents? A Two-Agent Ablation Study on Real Repositories,” arXiv preprint arXiv:2607.27250, 2026.
- [37] Z. Nussbaum, J. X. Morris, B. Duderstadt, and A. Mulyar, “Nomic Embed: Training a Reproducible Long Context Text Embedder,” arXiv preprint arXiv:2402.01613, 2024.
- [38] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. W. Cohen, R. Salakhutdinov, and C. D. Manning, “HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering,” arXiv preprint arXiv:1809.09600, 2018.
- [39] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, “Gorilla: Large Language Model Connected with Massive APIs,” arXiv preprint arXiv:2305.15334, 2023.
- [40] D. Wu, H. Wang, W. Yu, Y. Zhang, K.-W. Chang, and D. Yu, “LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory,” arXiv preprint arXiv:2410.10813, 2024.
- [41] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, “ τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains,” arXiv preprint arXiv:2406.12045, 2024.
- [42] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, “MuSiQue: Multihop Questions via Single-hop Question Composition,” *Transactions of the Association for Computational Linguistics*, 2022.

VENKATA SANGARAJU (Member, IEEE) is an Independent Researcher working on reliability and governance of enterprise AI agents.

SUDHIR VISSA is with SAGE7 AI, Georgetown, TX, USA, working on enterprise AI agent architectures and data governance.